Energy M&A Intelligent Scraper

Architecture, Workflow, Features & Strategic Roadmap

Confidential | Board of Directors Presentation | April 2026

Table of Contents

1.	Executive Summary
2.	Project Overview & Business Case
3.	System Architecture: End-to-End Pipeline
4.	Python Module Architecture: File Relationships
5.	Data Sources: What We Monitor
6.	AI Classification Engine: How Deals Are Verified
7.	Key Features & Technical Capabilities
8.	Output Structure: The Excel Report
9.	Pros & Cons: Honest Assessment
10.	Roadmap & Next Steps
11.	Technical Specifications

1. Executive Summary

The Energy M&A Intelligent Scraper is a fully automated, AI-native deal intelligence platform purpose-built for the global energy sector. It continuously monitors 84 curated news sources ranging from international wire services (BusinessWire, GlobeNewswire, PR Newswire) to specialised energy trade press (Upstream Online, Natural Gas Intelligence, Hart Energy) and SEC Government filings, extracting, verifying, and classifying every merger, acquisition, joint venture, and divestiture announcement into a structured, multi-sheet Excel report.

The AI-verification gate is the system's defining strength: every candidate deal passes through a four-tier reasoning pipeline powered by Groq's Large Language Model infrastructure before it enters the final output. This delivers near-zero false positives while maintaining near-complete recall of genuine transactions. The system runs fully autonomously, generates an executive narrative summary, and costs nothing in recurring API fees.

Parameter	Value / Description
Daily Deal Volume	40-80 verified M&A deals per run across all energy sub-sectors

Source Cove

2. Project Overview & Business Case

The Problem We Are Solving

-> Manual deal monitoring across 80+ energy news sources takes an analyst 4-6 hours every day.

-> Deals are frequently missed because they appear in niche trade publications or wire service subpages.

-> No standardised system exists to route deals into correct categories: Upstream vs. Midstream vs. Renewables.

-> Manual tracking introduces human error and inconsistency across team members.

-> Competitor monitoring is reactive, not proactive: deals are discovered days after announcement.

The Solution

-> Fully automated scraper covering 84 sources in under 40 minutes, with zero manual touch.

-> AI-verified output: only genuine M&A transactions appear in the final report.

-> Structured classification into 5 O&G sub-sectors + Mining, JV/Partnerships, and Reports sheets.

-> Self-learning: the system remembers previously confirmed companies and refines accuracy per run.

-> Zero ongoing cost: Free-tier AI APIs + local SQLite caching for all repeat queries.

Target Users & Use Cases

-> M&A Analysts: Daily digest of confirmed energy transactions across the full value chain.

-> Strategy Teams: Competitor activity tracking and sector consolidation trend analysis.

-> Executives: One-page AI-generated market narrative summarising the day's deal landscape.

-> Investment Teams: Rapid identification of deal flow in specific geographies or sub-sectors.

3. System Architecture: End-to-End Pipeline

The scraper operates as a linear, asynchronous pipeline. Every article from every source passes through each stage in sequence. The pipeline is architecturally designed to discard noise as early (and cheaply) as possible: expensive AI API calls are only triggered after all deterministic, zero-cost filters have already passed the article.

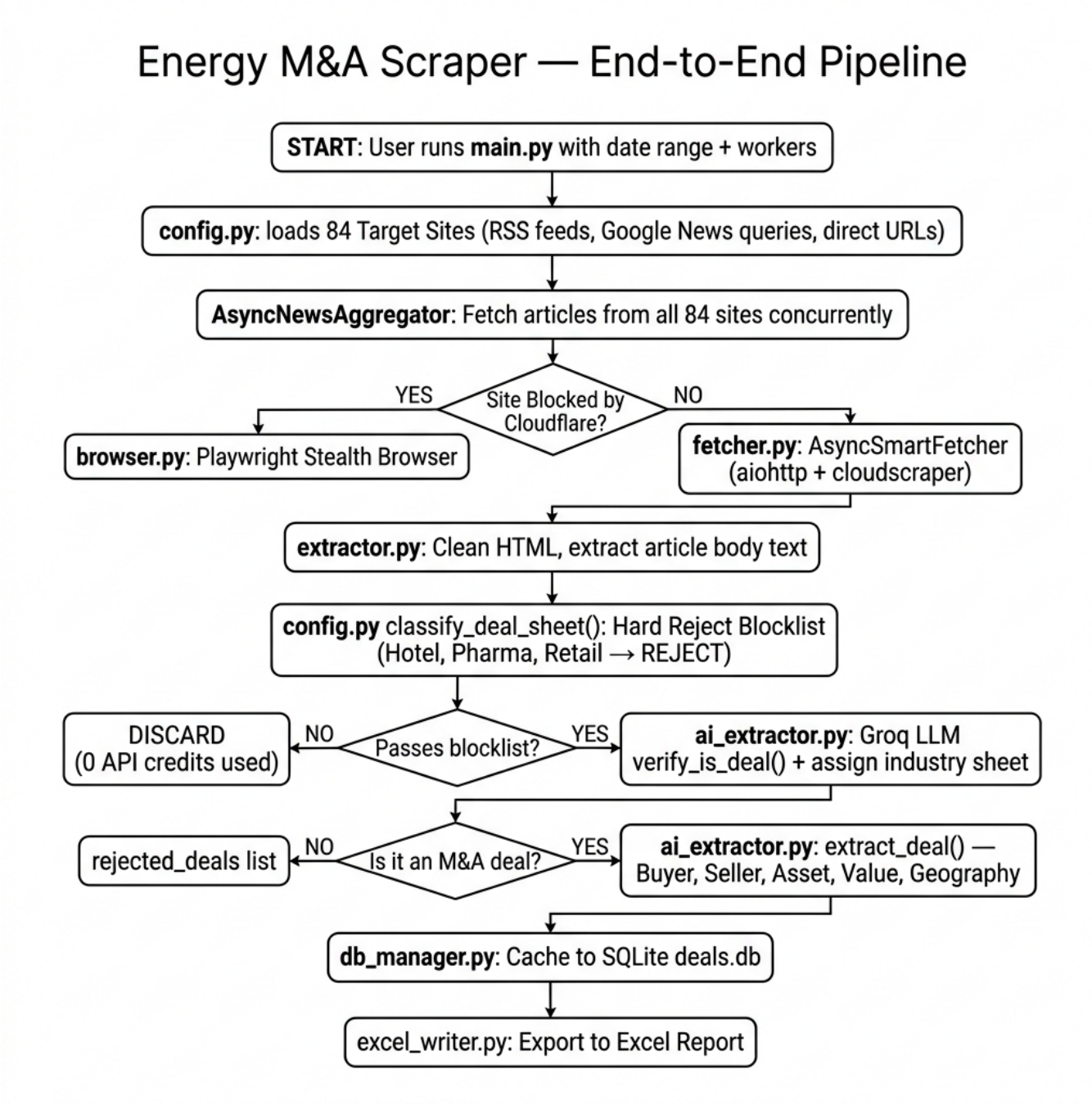

Figure: End-to-End Pipeline: From Command Line Input to Excel Report Output

Stage-by-stage explanation:

-> Stage 1 (Input): User specifies a date range and number of parallel workers via command line.

-> Stage 2 (Config): config.py provides all 84 site definitions including URLs, RSS feeds, and GN queries.

-> Stage 3 (Concurrent Fetch): All 84 sites are scraped simultaneously via Python asyncio event loop.

-> Stage 4 (Access Layer): Unprotected sites use aiohttp; Cloudflare-protected sites use Playwright Stealth.

-> Stage 5 (Extraction): HTML is cleaned and article body text is extracted using trafilatura.

-> Stage 6 (Hard Reject): Non-energy phrases (hotel, pharma, retail) are caught here. Zero API cost.

-> Stage 7 (AI Gate): Groq LLM verifies each article and assigns an industry sheet in one JSON call.

-> Stage 8 (Entity Extraction): Buyer, Seller, Asset, Deal Value, Geography are structured by AI.

-> Stage 9 (SQLite Cache): Each deal is persisted to deals.db for deduplication and re-run caching.

-> Stage 10 (Excel Export): All verified deals are written to the final structured xlsx report.

4. Python Module Architecture: File Relationships

The project is composed of 11 Python modules, each with a single clearly-defined responsibility. This separation-of-concerns architecture makes the system maintainable, debuggable, and straightforward to extend with a new feature or data source.

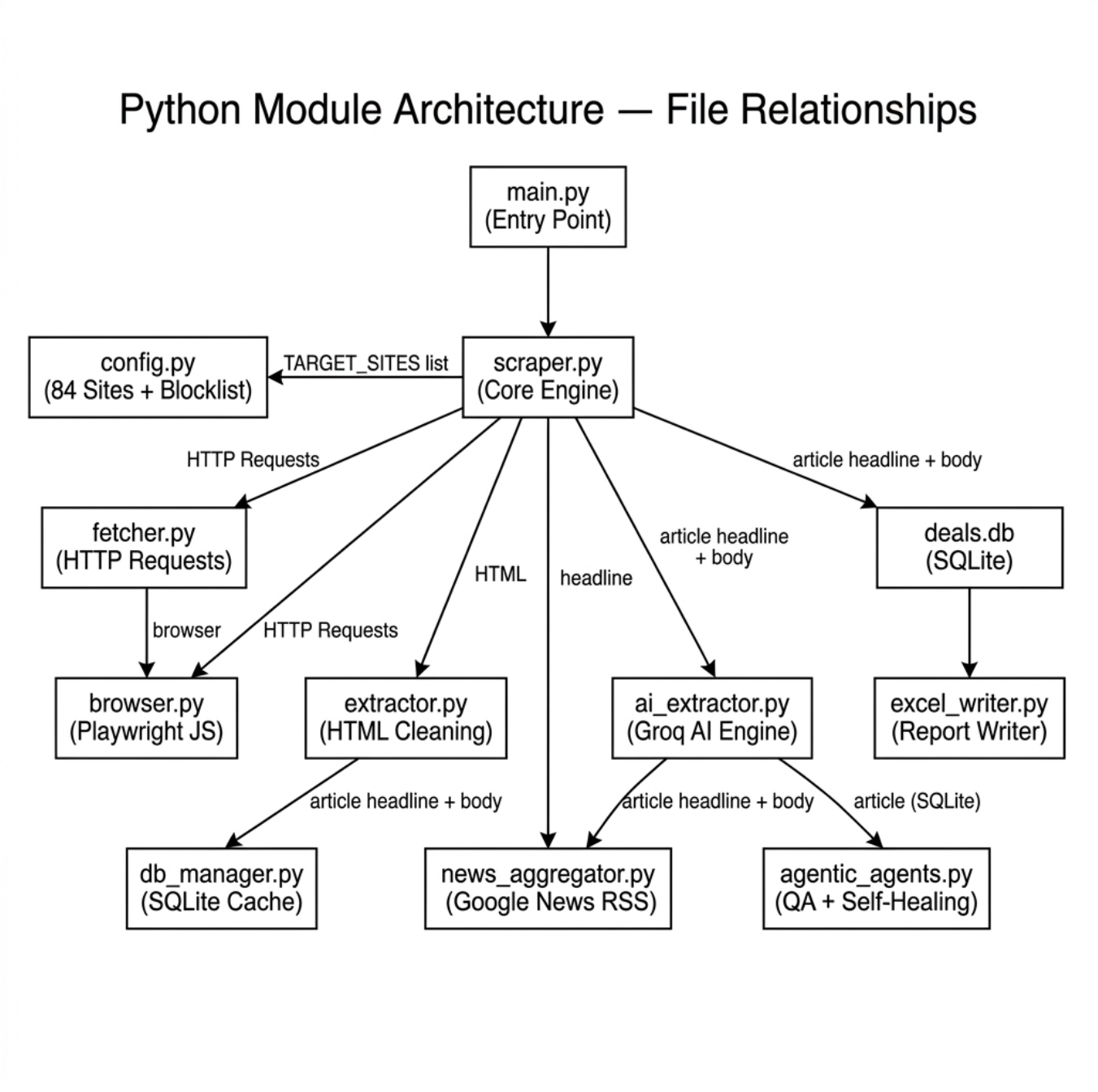

Figure: Python Module Dependency Graph: Data flows between project files

Parameter	Value / Description
main.py	Entry point. Parses CLI arguments, initialises all modules, starts scraper.run().

scraper.py

5. Data Sources: What We Monitor

The scraper monitors 84 precisely targeted endpoints across 6 strategic source categories. Each site is configured with its optimal access method: RSS feeds, Google News queries, or Playwright browser rendering. A single publisher can contribute multiple endpoints (e.g., RenewablesNow has 6 section-specific RSS feeds covering Solar, Wind, Storage, Hydro, and Other Renewables).

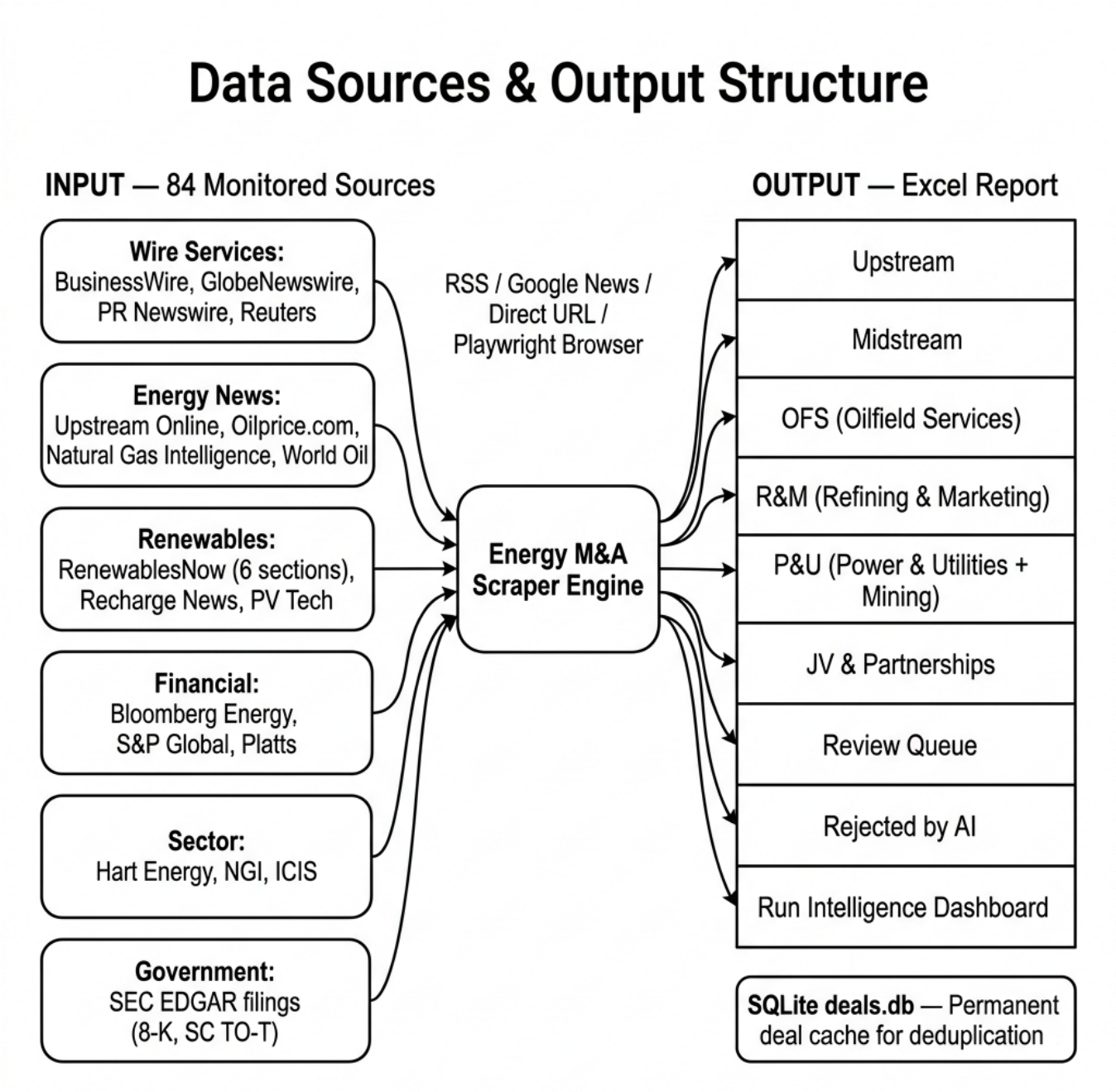

Figure: All 84 Input Sources (left) and the Structured Excel Output (right)

Parameter	Value / Description
Wire Services (high vol.)	BusinessWire, GlobeNewswire, PR Newswire, Reuters Newswire, Nasdaq GlobeNewswire

Energy Trad

6. AI Classification Engine: How Deals Are Verified

The AI classification engine is the system's core differentiator. It operates in four tiers, architecturally ordered so that the most computationally expensive operations (LLM API calls) are only invoked after all cheaper, deterministic methods have already screened the candidate article.

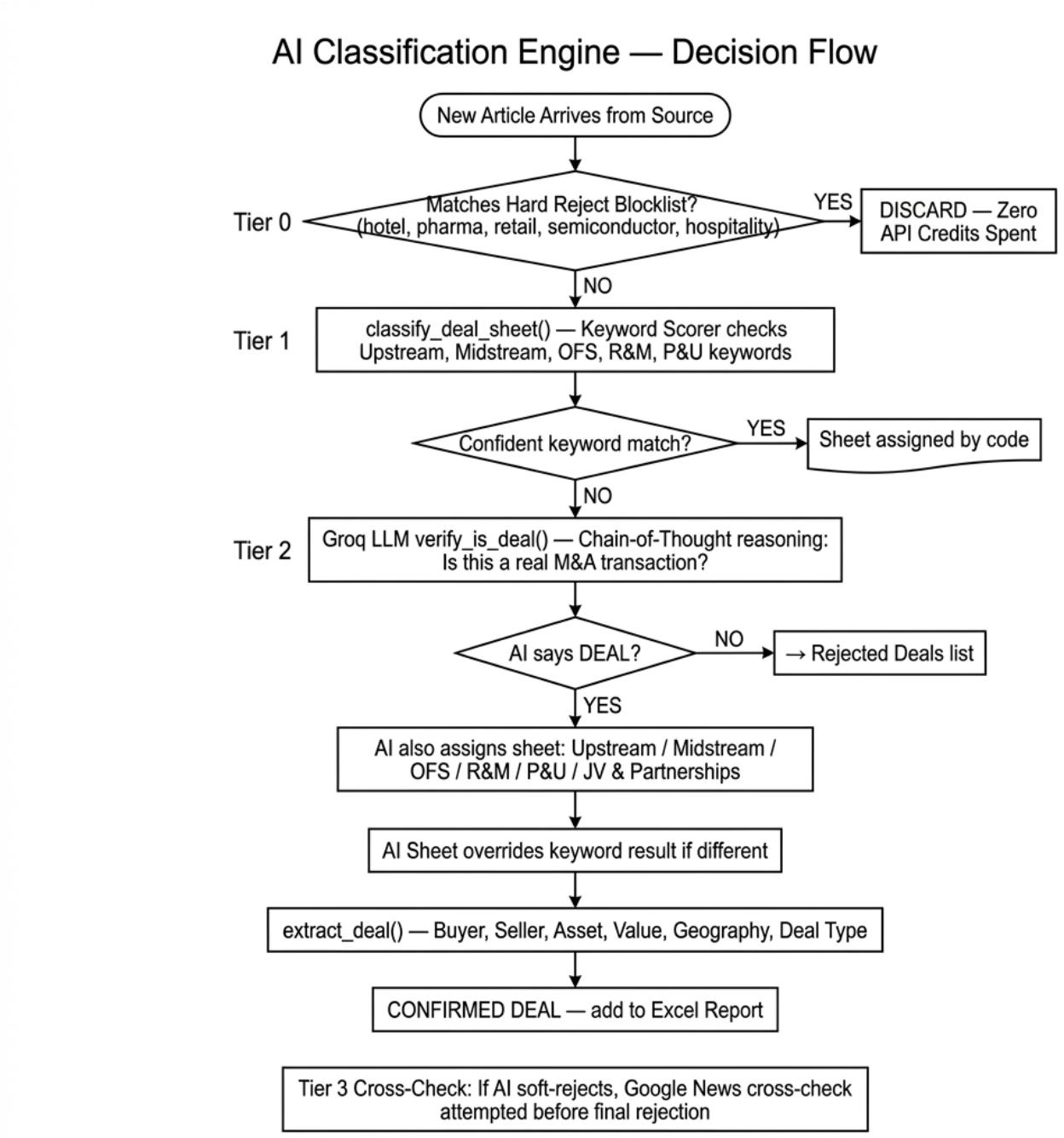

Figure: Four-Tier Decision Pipeline: Verification and Sheet Assignment Logic

-> TIER 0 - Hard Reject Blocklist: Python set lookup. If the headline contains 'hotel', 'pharma', 'biotech', 'semiconductor company', the article is discarded instantly. Zero API calls, zero cost. This single mechanism eliminates ~30% of all inbound articles before any AI is invoked.

-> TIER 1 - Keyword Classifier: Scores the article against 200+ energy keywords organised by sheet (Upstream: 'acreage', 'E&P', 'well'; Midstream: 'pipeline', 'LNG terminal'; P&U: 'solar farm', 'wind'). If a confident match is found,

the sheet is pre-assigned before the LLM call.

-> TIER 2 - Groq LLM Gate: verify_is_deal() sends the headline + article snippet to Groq LLM. The AI performs chain-of-thought reasoning and returns is_deal (yes/no), confidence (0-1.0), reasoning, reject_reason, AND the industry sheet assignment - all in a single structured JSON response. One API call does both the verification and the classification.

-> TIER 3 - Cross-Check Rescue: If the AI soft-rejects an article (uncertain), a Google News cross-check is automatically triggered. If 5+ independent sources confirm the deal exists, it is rescued from the rejected pile with 0.60 confidence and placed in the Review Queue.

7. Key Features & Technical Capabilities

Parameter	Value / Description
Concurrent Async Scraping	Python asyncio scrapes all 84 sources simultaneously. A serial run of 8+ hours is compressed to ~35 minutes.
Playwright S	

8. Output Structure: The Excel Report

The final output is a structured .xlsx workbook automatically named with the run timestamp (e.g., Energy_MA_Report_Async_20260331_130145.xlsx). It contains 9 worksheets, each colour-coded by industry. Every row represents one verified deal with all key fields populated by the AI.

Parameter	Value / Description
Upstream	Oil & gas E&P deals: field acquisitions, working interests, basin entries, E&P company buyouts
Midstream	

Fields included for each deal row:

-> Headline | Buyer | Seller | Asset/Target | Deal Value (USD-normalised) | Deal Type | Deal Status

-> Geography | Industry | Sector | Strategic Rationale (AI 1-sentence summary)

-> Confidence Score (0.00-1.00) | Source Publication | Direct Link to Article | Date

9. Pros & Cons: Honest Assessment

Strengths

-> Near-zero cost: Built entirely on free-tier APIs (Groq, Google News RSS, SEC EDGAR, open.er-api.com).

-> Highly configurable: Adding a new source is one dictionary entry in config.py. No code changes needed.

-> Intelligent noise filtering: Hard-reject blocklist eliminates off-sector articles at zero API cost.

-> Transparent AI reasoning: Every decision includes a full 'reasoning' field for analyst auditability.

-> Self-improving: The feedback loop and deal memory mean classification accuracy increases each run.

-> Zero re-run cost: SQLite caching means running the same date range twice is completely free.

-> Full sector coverage: Upstream through Renewables, Mining, and JVs in one unified tool.

-> Legally mandated coverage: SEC EDGAR integration ensures no public company deal is ever missed.

Limitations & Trade-offs

-> Hard Paywalls: Sites like Financial Times Energy, Upstream Online, and Recharge News sit behind hard paywalls. The scraper can only see headlines from these sources, not full article bodies.

-> Groq Rate Limits: The free Groq API allows 30 requests per minute per model. During high-volume runs, HTTP 429 rate-limit errors are common, triggering model rotation and adding processing time.

-> Batch-Based, Not Real-Time: The scraper requires manual execution with a date range. It is not yet a continuous streaming system; it produces a point-in-time snapshot report.

-> Language Handling: Non-English sources (Russian Neftegaz, Spanish Expansion Energy) require AI translation before classification, adding one additional LLM call per article.

-> No Proxy Rotation: The system currently uses the host machine IP. Aggressive scraping of a single protected site could trigger a temporary IP block (mitigated by access-method fallbacks).

-> Playwright Resource Cost: Sites requiring headless browser rendering are slower and consume more RAM (~350-500MB peak) than simple HTTP requests.

-> Configuration Maintenance: The 84-site config requires periodic review as RSS feed URLs change, paywalls are added, and new publications emerge.

10. Roadmap & Next Steps: Ultra Pro Max

Phase 1: Infrastructure (0-3 months)

-> Proxy Integration: Connect a rotating residential proxy pool (BrightData or ScrapFly) to browser.py to prevent IP blocks and maximise access to protected publications.

-> Message Queue: Migrate from asyncio batches to Redis + Celery workers so a frozen Playwright browser instance cannot block the entire pipeline.

-> Scheduled Execution: Deploy as a Windows Scheduled Task running automatically at 07:00 daily, emailing the Excel report to the distribution list.

-> Sitemap Integration: Add ultimate-sitemap-parser as a Tier 4.5 fallback for sites that block RSS.

Phase 2: Accuracy (3-6 months)

-> Semantic Deduplication: Use sentence-transformers embeddings to detect near-duplicate deals ('Shell buys LNG asset' vs 'Shell acquires LNG terminal') beyond URL and exact-text matching.

-> Company Entity Normalisation: Map extracted Buyer/Seller names to LSEG or Yahoo Finance IDs to standardise entity names and auto-populate ticker symbols in the report.

-> Domain-Specific Classifier: Fine-tune a small DistilBERT model for energy deal sheet classification, replacing Groq LLM Tier 1 calls with a local ~3ms inference model.

Phase 3: Analytics Platform (6-12 months)

-> Streamlit Dashboard: A live web interface showing deal flow trends, acquirer frequency maps, geographic heat maps, and a UI for analysts to correct AI mistakes.

-> PostgreSQL Migration: Replace local SQLite with a hosted database supporting multi-user access.

-> Weekly Intelligence Digest: Auto-generated PDF emailed to senior stakeholders every Monday morning, summarising the week's transactions and identifying emerging M&A themes.

-> API Access Layer: RESTful API allowing other internal tools to query the deal database programmatically.

11. Technical Specifications

Parameter	Value / Description
Language	Python 3.13
Async Fram	